

Archify 小白指南

把一句话/一个仓库变成可探索的交互式架构图 —— Agent Skill 形态的画图神器 (tt-a1i/archify, 76k+ stars)

GitHub 76.2k stars

MIT 开源

v3.0.1 (2026-09)

GitHub Trending 周榜 #1

产物: 单文件交互 HTML

Q1 Archify 到底是什么？一句话说清

你把一句话需求（或一个真实仓库）丢给 AI 编程助手，Archify 让它产出一张**能点、能搜、能走路径、能导出 PNG/SVG 的交互式图**——不是一张死图片，而是一个自包含 HTML 小应用。

它的本体不是传统软件，而是一个 **Agent Skill**：一个 `SKILL.md` 说明书 + 一套本地 CLI 校验工具。AI 读完说明书，按规范写一份 JSON，再用 CLI 渲染成交互 HTML。**没有服务器、没有账号、没有联网依赖**，全部发生在你本机。



Q2 为什么能拿 76k stars? 它踩中了什么痛点

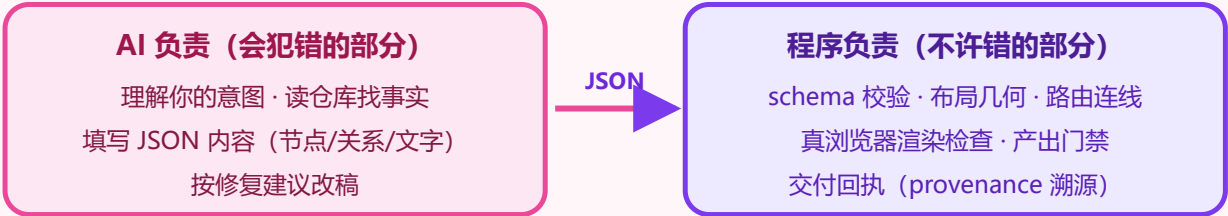
画架构图的老三样各有硬伤，Archify 用"AI 生成 + 程序校验"一次全绕开：

工具	怎么画	痛点
draw.io / Visio	手动拖拽	慢；系统一改图就烂；无法跟着代码走
Mermaid	写文本代码生成	样式丑、布局失控、只有静态图不能交互
直接让 AI 画 SVG	AI 自由发挥	会撒谎 ：画布外溢出、线穿节点、画出代码里不存在的调用
Archify	AI 按严格 schema 写 JSON → CLI 校验 → 真浏览器检查	美观可控 + 事实可查 + 产物可交互

本质洞见：**不让 LLM 自由画坐标，而是让它填一张有类型约束的表**，几何由确定性渲染器算——这正是"约束式生成"战胜"自由式生成"的活教材。

Q3 "Agent Skill"形态怎么工作? 为什么不用装服务

SKILL.md 只有 107 行，是一个给 AI 看的**作业指导书**：先选图类型 → 按路由读对应 schema 和示例 → 直接写候选 JSON → 跑 `finalize`。AI 能力（理解需求、读代码）和程序能力（几何计算、校验）在此分工：



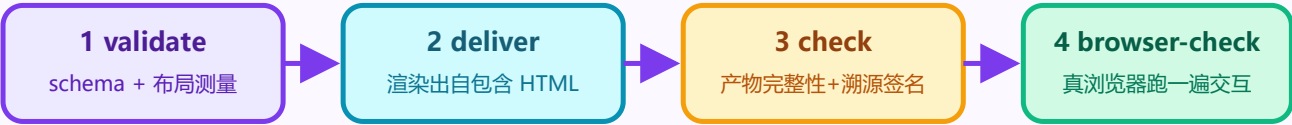
安装也只是一句话：`npx skills add tt-ai/archify -g`——把技能包注册给 Claude Code / Cursor / Codex CLI，之后对话里说"用 Archify 画一下....."即可。

Q4 五种图类型：typed JSON (JSON-IR) 长什么样

类型	用在哪	核心字段
<code>architecture</code>	系统组件 / 云拓扑 / 安全边界	nodes(带 type 语义) + relationships + boundaries
<code>workflow</code>	流程 / 审批 / CI-CD / 工具调用	lanes(泳道) + nodes + edges (本指南实验用它)
<code>sequence</code>	API 调用链 / 请求生命周期	participants + messages(带 y 坐标) + activations
<code>dataflow</code>	管道 / ETL / 血缘	sources → transforms → consumers
<code>lifecycle</code>	状态机 / 重试 / 终态	states + transitions

关键设计：**节点的 type 不是形状而是语义**——只能是 frontend / backend / database / cloud / security / messagebus / external 七种（实验 2 实测：填 `process` 直接被 schema 拒绝）。渲染器按语义上色、定形状、定图例，所以**每张图风格一致、颜色即含义**。

Q5 finalize 四道门禁：为什么图能“保证不烂”



任一门失败 → 退出码非 0 → AI 读回执自动修稿重跑（有限次）

源码实证（`bin/finalize.mjs` 第 21 行）：四个阶段是常量 `FINALIZE_STAGES = ['validate', 'deliver', 'check', 'browser-check']`，全过才发“通过回执”。**browser-check 会真的启动 Chrome/Edge 无头渲染产物**，不是模拟。AI 的稿件被当成“候选（candidate）”，程序才是终审法官。

Q6 产物是“自包含交互 HTML”——读者能玩什么

- **缩放分级显示**：100% 以下自动降为地图模式，175% 以上露出全部细节（Reading Depth）；
- **Node Finder 搜索**：按标签或 ID 找节点并聚焦；**Semantic Passport**：点节点弹出它的上下游事实卡 + 可复制深链；
- **Route Probe 路径探测**：选两个端点，高亮真实有向路径（拒绝几何脑补）；**Reach**：看某节点的全部上下游可达集；
- **导出**：PNG / JPEG / 双主题 SVG / 带描边动画的 WebM，还有 Route / Reach 分享卡；
- **trace 动画**：可选的请求流动画，静态永远是兜底（尊重 prefers-reduced-motion）。

实验 1 实测：官方示例各 744 KB，外部 script/link/img 引用 **0 个**——发微信、传内网、离线双击全能开，查看方无需装任何东西。

Q7 画真实仓库的图，怎么保证 AI 不撒谎

这是 archify 最"工程狠"的部分 (`references/repository-authoring.md`)：

- **钉死身份**：图必须记录 `git rev-parse HEAD` 的 40 位 commit SHA + 凭证剥离的 origin URL，写进 `meta.repository`；
- **证据锚定**：每条"谁调用谁"的关系都要附源码位置 (`sourceReferences`)，校验器对照**该 commit 的字节**验证，工作区未提交的改动不算数；
- **语义检查**：candidate 里可写 `semanticChecks` ("缓存只在 miss 后才出现"这类断言)，交付时逐条核对；
- **诚实边界**：只画读到的切片，禁止为了图好看补一条"看起来该有"的线。

一句话：图和代码之间有据可查，职责声明不构成 I/O 证据——"maintains tasks" 这种注释不能证明真的直接读写文件。

EXP 动手实验：亲手走一遍 Archify 的核心链路

实验材料在 `F:\reminder\archify\exercise\`。主题：给本仓库自己的"拾遗学习流水线"画一张 workflow 图。

步骤 1 · 克隆并摸底 (✓ 已真实运行)

```
git clone --depth 1 https://github.com/tt-ali/archify repo
# 验证: git -C repo log --oneline -1 # → d5a1333
```

目的：拿到技能包本体。实现：repo\下 745KB 级 skill 包 (SKILL.md + schemas + renderers + bin)。

步骤 2 · 静态体检官方产物，验证"自包含"承诺 (✓ 已真实运行)

```
python exercise/exp1_artifact_check.py
```

目的：不信广告，用脚本数外部依赖。✓ 预期结果 (实测)：两个官方示例各 744KB，外部 script/link/img = 0，主题切换 / canvas 导出 / Route Probe / Node Finder / 键盘可达 / hashchange 深链 / reduced-motion 全部命中，每个节点带原生 <title> 无障碍提示。

步骤 3 · 自己写候选 JSON 并用官方 schema 校验 (✓ 已真实运行，含反例)

```
python exercise/exp2_schema_validate.py
# 用 jsonschema 4.26 + 官方 schemas/workflow.schema.json
# 校验自编的 shiyi-pipeline.workflow.json (拾遗流水线，3泳道8节点8边)
```

✓ 预期结果 (实测三段变化)：

- ① 首跑 **7 个错误**——节点 type 填了流程图形状词 (process/decision)，schema 只认七种语义类型；
- ② 按语义改 (审批门→security，远端→cloud...) 后正例 **0 错误**；
- ③ 反例测试：删 lanes → 被"required property"拦下；type=magic-box → 被 enum 拦下。**门禁真的拦得住。**

步骤 4 · finalize 生成正式交互 HTML (⚠ 已给出，本机未验证)

```
node repo/archify/bin/archify.mjs finalize workflow exercise/shiyi-pipeline.workflow.json exercise/shi
```

⚠ **诚实声明**：本机环境的安全策略禁止执行刚克隆的外部代码，此命令**未真实运行**，产物未生成。步骤 1-3 已覆盖该技能的核心机制 (schema 约束 + 产物形态)，此命令的效果 (四道门禁回执 + 交互 HTML) 由源码与官方 gallery 佐证。你在自己的终端直接运行即可补全。

红线教训：本任务源推文是"保存这份清单"式带货帖，7 个仓库中 2 个根本不存在（omnivoice-studio、deepseek-ai/dsh）。任何仓库清单先逐个验证再学。

Q8 我能拿它干什么？能力清单

- **装进自己的 Claude Code**： `npx skills add tt-ai/archify -g`，以后说"用 Archify 画××的架构"就有高质量交互图；
- **升级学习流水线**：learn-project 产出的 PDF 是静态 SVG——今后架构/流程类示意图可先用 archify 出交互版，截图/导出 PNG 再进 PDF，双份产出；
- **读陌生仓库提速**：clone 完让 agent"用 Archify + repo 溯源画主链路"，比人工翻代码快一个量级，且每条边可溯源；
- **Mermaid 资产升级**：存量 Mermaid flowchart/sequence/state 图可转成交互版（读拓扑重写 JSON，不搬样式）；
- **团队分享**：单文件 HTML 直接扔群里，对方双击就能搜节点、走路径，不用装任何东西。

quality_profile 二档：`showcase`（默认，杂志级排版）与 `standard`（信息密度优先）。另有 `migrate` 命令负责旧 JSON 升版，`compare` 命令对比两个版本的架构差异。

Q9 和自己已知概念怎么连？

- **vs Mermaid**：Mermaid 是"文本→静态SVG"，布局算法黑盒不可控；archify 是"类型化 JSON→校验→交互 HTML"，AI 只填内容不碰几何；
- **vs MCP 工具**（知识库已有概念）：MCP 给 AI 调用外部能力，Skill 给 AI 注入作业指导书——archify 两者都不占，纯提示词+本地CLI，零服务零账号；
- **vs 让 LLM 直接画 SVG**（本仓库 PDF 流水线现状）：LLM 画图坐标误差大、无校验；archify 证明"生成物必须有确定性验收"这条路，其 finalize 门禁思想可以反哺我们 PDF 产出的质量检查环节。

Archify 小白指南 · 拾遗任务 416bb314 · 2026-10-03 · 实验 1/2 真实运行，实验 3 因本机安全策略未执行（命令已给）